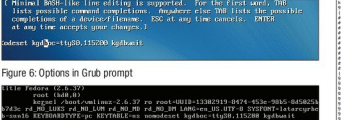

Figure 6: Options in Grub prompt

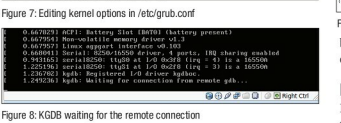

Figure 7: Editing kernel options in /etc/grub.conf

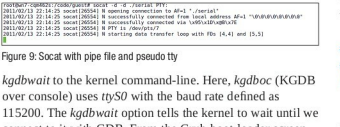

Figure 8: KGDB waiting for the remote connection

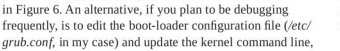

Figure 9: Socat with pipe file and pseudo tty

kgdbwait to the kernel command-line. Here, *kgdboc* (KGDB over console) uses *ttyS0* with the baud rate defined as 115200. The *kgdbwait* option tells the kernel to wait until we connect to it with GDB. From the Grub boot-loader screen, press '*e*' and append the options to the kernel line, as shown in Figure 6. An alternative, if you plan to be debugging frequently, is to edit the boot-loader configuration file (*/etc/grub.conf*, in my case) and update the kernel command line, as shown in Figure 7. This second method requires a reboot of the VM to activate the new kernel options, if you haven't edited at the GRUB prompt.

Booting with KGDB options

Once you have these options in the kernel command-line and boot from it, the kernel boots till it gets to the stage where it waits for the remote GDB connection over the (virtual) serial port, as shown in Figure 8.

Linking the serial file on the host to the pseudo-terminal

We need to use *socat* to do this linking, with the following command:

```
# socat -d -d /code/guest/serial PTY:
```

Here, *PTY:* is the pseudo-terminal, and '*/code/guest/serial* ' is the virtual serial port pipe file created by VirtualBox on my host machine as per the VM settings done earlier. When this command is run, it returns the pseudo-terminal number that's allocated (in my case, */dev/pts/7*), as shown in Figure 9.

It's important to remember that you should not terminate the *socat* command; it needs to be running in the background for us to be able to use the pseudo terminal, else it breaks the stream.

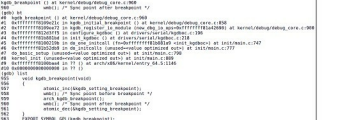

Figure 10: GDB connected to guest OS

Firing up GDB

Enter the kernel source directory on the host, and start GDB, telling it to connect to the remote target, which is the pseudo-terminal number returned by *socat*:

```
# cd linux-2.6.37
# gdb ./vmlinux
(gdb) target remote /dev/pts/7
```

This connects us to the waiting Linux kernel session in the VM. If we type '*continue*' at the *gdb* prompt, we will see booting resume in the VM's guest OS. To be able to get back the *gdb* prompt on the host, you need to run the following command in the guest:

```
# echo g > /proc/sysrq-trigger
```

This will break the running session, and give you control in GDB. This can be used to insert break-points and do other debugging operations, like those seen in Figure 10. If you need to debug a kernel module, insert the module in the guest, obtain the .text address of the module (*/sys/module/<module_name>/sections/.text*) and use it as an argument for the GDB command '*add-symbol-file*'.

As you can see, with the above set-up, a great deal of control can be achieved in debugging the live kernel. END

Footnotes

[1] Oracle's VirtualBox.org website
[2] KGDB homepage http://kgdb.linsyssoft.com/

By: Surya Prabhakar

The author is an engineering advisor in the Product Group at the Dell India R&D Centre, Bengaluru, and has eight years of experience in Linux. He spends most of his time hacking and playing around with Linux. He can be reached at *surya_prabhakar@dell.com*.